Data Collection and Preprocessing

Project: Evaluating Forecast-Based Alerts for Cloud Infrastructure Monitoring

Student: Mamadou Kaba - COEN 6314

Data milestone | November 1, 2026

Data obtained

I downloaded all 17 time series in the Numenta Anomaly Benchmark (NAB) realAWSCloudwatch subset and its combined_windows.json annotations. These are historical, single-metric AWS monitoring recordings, including CPU, network, disk and request metrics. The source supplies an MIT license, retained with the download; its copyright and permission notice must accompany reused material. The download manifest records the exact source revision, file sizes and SHA-256 checksums so the same data can be obtained again.

Audit result	Observed value
Time series / original readings	17 / 67,740
Original CSV size	1,855,225 bytes
Unique timestamps, counted within each series	67,718
Missing or non-finite values	0
Excess duplicate timestamp rows	22
Intervals differing from the training-derived five-minute cadence	19
Annotated anomaly windows	30
Unique readings inside annotation windows	6,312 (9.32%)
Rows with complete, regular feature history	67,308

The forecasting target is the reading at time t, using only earlier readings from that series. Alerts are produced after the new reading arrives. Window annotations are separate evaluation labels, not regression targets or predictors. Window membership is imbalanced (about 9.32% inside versus 90.68% outside), but it is not a verified operational incident rate. NAB windows provide tolerance around annotations; they do not establish incident duration or failure onset. These short historical recordings are not a representative sample of current production infrastructure, and five-minute sampling cannot resolve faster changes. Timestamp timezone is not specified in the CSVs; timestamps are preserved without assuming UTC or local time.

Preparation decisions

I sorted each series by timestamp and collapsed duplicate timestamps using the median value. Two files contain 11 excess rows each: ec2_disk_write_bytes_1ef3de has identical duplicate values, while ec2_network_in_5abac7 has conflicting values at one timestamp. The median avoids counting one timestamp repeatedly and reduces the influence of an arbitrary duplicate. Raw files are preserved. This assumes same-timestamp records are available together; arrival order is not supplied by the benchmark.

I retained unusual values, including zeros, because removing or clipping them could erase the anomalies being studied. I did not interpolate gaps or resample. Rows lacking a complete, finite, regularly spaced history are ineligible for prediction; processing resumes after 12 consecutive prior readings are available. This removes 410 rows through initial warm-up and interrupted histories. All three methods will use the same eligible timestamps.

Initial predictors are lags of 1, 2, 3, 6 and 12 readings, plus the mean and population standard deviation of the preceding 6 and 12 readings. They describe recent level, change and variability over five minutes to one hour. Every predictor excludes the current target reading. These are initial, fixed choices; any later selection will use development data only. No scaling or imputation is fitted. Random Forest does not require feature scaling, and separate models are planned for each series to avoid mixing metric units. Sampling cadence is estimated from training timestamps only; any later learned transformation will also be fitted on training data only.

Chronological split and leakage control

Within each series, the first 20% of unique timestamps form training data, the next 40% development (validation) data and the last 40% test data. There is no random shuffle or pooling across series. This split follows an annotation-count feasibility audit conducted before model fitting, rather than a comparison of test results.

Period	Usable forecast examples	Complete windows	Series with complete windows
Training	13,296	0	0
Development	26,956	13	11
Test	27,056	13	11

This split prioritizes retaining annotated events for development and testing at the cost of limited training history. Each series has 236-931 usable training examples, most commonly 794. This permits the planned experiment but may limit the patterns Random Forest can learn. The split is now fixed and will not be changed in response to model results.

Zero complete training windows does not establish anomaly-free training: a boundary-crossing window can overlap it. The regression model learns metric values and does not require examples of both anomaly classes. Earlier observed readings may supply history at the start of later splits, as in sequential monitoring; later values cannot alter earlier predictors. Model parameters will be fitted on training data, with settings and alert thresholds selected on development data. Test annotation locations and counts have been inspected solely for split feasibility. No test model performance has been computed. Test annotations will not guide subsequent modelling and will be used for scoring only after settings are frozen.

Four boundary-crossing windows are recorded separately and excluded from evaluation, including their timestamps in scoring exposure. Their readings remain in the observed history; they are not removed from model inputs. Exposure is the number of eligible, non-excluded readings times the training-derived sampling interval, rather than unobserved wall-clock gaps. The separate evaluation mask supports consistent handling for all methods.

Remaining work and limitations

The downloader and preprocessing pipeline ran successfully. Four automated checks passed, covering predictor causality, gap handling, missing-value handling and chronological separation of generated splits. Detailed per-series quality and split tables are saved alongside this report. No dataset-access problem remains.

The main unresolved limitation is the small, uneven event count: only 13 complete test windows remain across 11 series. I will retain all series for false-alert analysis, report event detection as undefined where no windows exist, and present per-series results plus equal-weight summaries with their denominators. The evidence will support an exploratory comparison, not a strong general claim about production reliability. Next, I will implement the three methods and shared episode-matching rules, select settings on development data, then run one final test evaluation.

Source: https://github.com/numenta/NAB (revision recorded in config/data_manifest.json).